Performance Evaluation of Document Databases

Vinay Jaikumar Gupta
Illinois Institute of Technology, Chicago, IL
vgupta31@hawk.iit.edu
A20554266

***Abstract*—In the era of big data, where applications face the challenge of handling millions of queries, the performance of document databases, commonly referred to as NoSQL databases, becomes crucial. This study conducts a thorough performance evaluation of four prominent NoSQL document databases MongoDB, CouchDB, CosmosDB, and DynamoDB under the stress of millions of queries. The research focuses on assessing the databases' capabilities in managing and optimizing a high volume of requests, considering factors such as query latency, throughput, and scalability. Through rigorous benchmarking, we explore strategies for optimizing the performance of these databases when subjected to a massive query load. The findings aim to provide practical insights and recommendations for developers and database administrators seeking to enhance the efficiency of document databases in scenarios involving millions of requests.**

***Index Terms*—MongoDB, CouchDB, DynamoDB, CosmosDB, CRUD Operation, NoSQL Document Database**

I. Introduction

In the rapidly evolving landscape of contemporary applications, NoSQL databases have become indispensable tools, specifically designed to scale efficiently, particularly when dealing with distinct data models. Among the diverse range of NoSQL databases, document stores have proven crucial, seamlessly handling Create, Read, Update, and Delete (CRUD) operations. Noteworthy is their adeptness in managing distributed data across multiple sites, aligning seamlessly with the intricacies of the three-tier internet architecture. Thanks to their persistent design and adaptable data structures, NoSQL databases can be effectively distributed across various machines, optimizing resource utilization.

This study focuses on evaluating the performance of four major document-oriented NoSQL databases—MongoDB, CouchDB, CosmosDB, and DynamoDB—especially in scenarios involving millions of queries. Document-oriented databases store data in JSON-like documents, offering flexibility without necessitating prior knowledge of keys and data types. Often described as schema-less databases, they enable retrieval based on fields, ranges, or regular expression patterns. Their horizontal scalability is achieved through efficient partitioning and replication strategies. Notably, they exhibit robust support for text processing, facilitating efficient regular expression-based text search queries. In light of the increasing demand for processing data at scale, comprehending how these databases perform under the pressure of millions of requests is crucial for developers and database administrators. This research seeks to uncover the strengths and limitations of these document databases, providing valuable insights for optimizing their performance in real-world applications grappling with substantial query loads.

II. Related Work

A. Brief About MongoDB

MongoDB, a popular NoSQL document-oriented database, has garnered significant attention in both academia and industry due to its flexible data model and scalability features. Numerous studies have explored various aspects of MongoDB, ranging from its architecture to performance evaluations and real-world use cases.

1) Architectural Overview:
Research has delved into MongoDB's architecture, detailing its core components, such as the storage engine, query planner, and sharding mechanisms. Understanding MongoDB's internal workings is crucial for developers and administrators seeking to optimize its performance in diverse scenarios.
2) Performance Benchmarks:
Several studies have conducted performance benchmarks to assess MongoDB's capabilities under different workloads. These evaluations often involve comparisons with other NoSQL databases, providing insights into MongoDB's strengths and areas for improvement. Metrics such as query latency, throughput, and scalability are commonly analyzed to gauge its suitability for specific use cases.
3) Indexing Strategies:
MongoDB's indexing strategies have been a focus of research, exploring the impact of various index types on query performance. Optimizing index usage is essential for enhancing the efficiency of data retrieval operations in MongoDB.
4) Real-world Use Cases:
Research has delved into real-world implementations of MongoDB across different domains, showcasing its practical applications. From content management systems to data analytics platforms, MongoDB's versatility in handling diverse data types and use cases has been explored.
5) Scalability and Sharding:
Scalability is a key feature of MongoDB, and studies have investigated its scalability limits and the effectiveness of sharding in distributing data across multiple nodes. Understanding how well MongoDB scales hori-

zontally is crucial for applications dealing with growing datasets.

6) Security Considerations:
   Security is a paramount concern for any database system, and studies have addressed MongoDB's security features, potential vulnerabilities, and best practices for securing MongoDB deployments. This is particularly crucial as MongoDB is widely used in web applications and other environments where data security is paramount.
7) Query Optimization:
   Understanding how MongoDB processes and optimizes queries is a focal point in some studies. Researchers explore query execution plans, indexing strategies, and performance implications to provide insights into optimizing query performance.

In summary, the body of related work on MongoDB is diverse, encompassing architectural insights, performance evaluations, practical implementations, and considerations related to scalability and security. This collective knowledge contributes to a comprehensive understanding of MongoDB's strengths and challenges in different contexts.

B. Brief About CouchDB

CouchDB, a prominent member of the NoSQL family known for its distributed architecture and schema-less design, has been the subject of various studies exploring different aspects of its functionality and performance.

1) Distributed Architecture:
   Researchers have focused on CouchDB's distributed architecture, examining how it manages data across nodes and clusters. Understanding the underlying mechanisms of CouchDB's distribution is crucial for developers and administrators seeking to deploy scalable and fault-tolerant systems.
2) Consistency Models:
   Studies have investigated CouchDB's consistency models, exploring its trade-offs between consistency and availability in distributed environments. This research sheds light on how CouchDB handles data consistency and how developers can tune these settings based on specific application requirements.
3) Replication Strategies:
   Replication is a fundamental feature of CouchDB, and research has explored its replication strategies and implications for data synchronization across multiple nodes. Examining CouchDB's replication mechanisms is essential for maintaining data integrity and availability in distributed deployments.
4) Performance Benchmarks:
   Various performance evaluations have been conducted to assess CouchDB's capabilities under different workloads and compare it with other NoSQL databases. Metrics such as read and write throughput, latency, and scalability are often analyzed to understand CouchDB's performance characteristics.
5) Use Cases and Applications:
   Researchers have explored real-world use cases and applications of CouchDB in different domains. From content management systems to healthcare applications, understanding how CouchDB fits into specific use cases provides valuable insights for practitioners.
6) Query Processing and Indexing:
   CouchDB's approach to query processing and indexing has been a subject of research, examining how it efficiently handles queries on large datasets. Optimizing query performance is crucial for applications relying on CouchDB for data retrieval.
7) Fault Tolerance and Reliability:
   CouchDB's fault tolerance mechanisms and reliability features have been investigated to understand how it copes with node failures and ensures data consistency in distributed settings. This is particularly important for applications that require high levels of reliability and resilience.
8) Security Considerations:
   Security aspects of CouchDB, including authentication and authorization mechanisms, have been explored in research studies. Understanding the security features and potential vulnerabilities of CouchDB is crucial for securing deployments in various environments.

In summary, the related work on CouchDB spans various dimensions, including its distributed architecture, consistency models, replication strategies, performance benchmarks, real-world applications, query processing, fault tolerance, and security considerations. This collective knowledge contributes to a comprehensive understanding of CouchDB's capabilities and considerations for effective use in different scenarios.

C. Brief about DynamoDB

DynamoDB, Amazon's fully managed NoSQL database service, has been a subject of various studies and evaluations, exploring different facets of its architecture, features, and performance characteristics.

1) DynamoDB Architecture:
   Researchers have delved into the architecture of DynamoDB, investigating its underlying mechanisms for distributed storage, partitioning, and replication. Understanding the architectural nuances is crucial for developers and administrators aiming to optimize DynamoDB deployments for scalability and reliability.
2) Consistency Models:
   Studies have focused on DynamoDB's consistency models, examining how it handles consistency and availability trade-offs in distributed environment. This research sheds light on the options available to developers when configuring DynamoDB for specific use cases.
3) Partitioning and Scaling:
   DynamoDB's scalability features, particularly its partitioning strategies and scaling mechanisms, have been analyzed in research studies. Exploring how DynamoDB

efficiently scales horizontally helps in designing systems that can handle varying workloads.

4) Performance Benchmarks:
   Various studies have conducted performance benchmarks to assess DynamoDB's capabilities under different workloads and compare its performance against other NoSQL databases. Metrics such as read and write throughput, latency, and scalability are common focal points in these evaluations.
5) Use Cases and Best Practices:
   Research has explored real-world use cases and best practices for DynamoDB adoption across different industries and applications. Understanding how DynamoDB fits into specific scenarios provides valuable insights for practitioners seeking optimal database solutions.
6) Query Processing and Indexing:
   DynamoDB's query processing capabilities and indexing strategies have been subjects of investigation, exploring how it efficiently handles queries on large datasets. Optimizing query performance is essential for applications relying on DynamoDB for data retrieval.
7) Cost Modeling and Optimization:
   Researchers have delved into cost modeling and optimization strategies for DynamoDB, exploring ways to manage costs effectively based on usage patterns and data access behaviors [cite]. Cost considerations are crucial for organizations seeking to maximize efficiency in their cloud deployments.
8) Security Features:
   Security aspects of DynamoDB, including authentication, authorization, and data encryption, have been explored in research studies [cite]. Understanding the security features and potential vulnerabilities of DynamoDB is crucial for securing applications and data stored in the database.

In summary, related work on DynamoDB spans various dimensions, including its architecture, consistency models, partitioning strategies, performance benchmarks, real-world applications, query processing, cost considerations, and security features. This collective body of knowledge contributes to a comprehensive understanding of DynamoDB's capabilities and considerations for effective use in diverse scenarios.

D. Brief about CosmosDB

CosmosDB, Microsoft's globally distributed, multi-model database service, has attracted attention from researchers and practitioners alike. Numerous studies have explored different aspects of CosmosDB, ranging from its architecture to performance benchmarks and real-world applications.

1) Multi-Model Capabilities:
   Researchers have examined CosmosDB's multi-model capabilities, which support document, key-value, graph, and column-family data models. Understanding how CosmosDB seamlessly integrates these models provides insights into its versatility for various use cases.
2) Global Distribution and Partitioning:
   Studies have focused on CosmosDB's global distribution and partitioning strategies, exploring how it efficiently replicates data across regions and distributes partitions for optimal performance. Global distribution is a key feature, and understanding its implementation is vital for designing globally scalable applications.
3) Consistency Models:
   The consistency models of CosmosDB have been a subject of research, with studies investigating how it manages consistency and availability trade-offs in distributed environments. Understanding the nuances of consistency options aids developers in configuring CosmosDB for specific application requirements.
4) Performance Benchmarks:
   Various studies have conducted performance benchmarks to evaluate CosmosDB's capabilities under different workloads and compare its performance against other databases. Metrics such as throughput, latency, and scalability are often assessed to gauge its suitability for various applications.
5) Real-world Use Cases:
   Research has explored real-world applications and use cases of CosmosDB across diverse industries and scenarios. This provides valuable insights for practitioners seeking to understand how CosmosDB can be effectively applied in different contexts.
6) Query Processing and Indexing: CosmosDB's query processing and indexing strategies have been subjects of investigation, exploring how it efficiently executes queries on diverse data models. Optimizing query performance is crucial for applications relying on CosmosDB for data retrieval.
7) Cost Modeling and Optimization: Studies have delved into cost modeling and optimization strategies for CosmosDB, examining ways to manage costs effectively based on usage patterns and data access behaviors. Cost considerations are crucial for organizations seeking to maximize efficiency in their cloud-based database deployments.
8) Security Features: Security aspects of CosmosDB, including authentication, authorization, and encryption, have been explored in research studies. Understanding the security features and potential vulnerabilities of CosmosDB is essential for ensuring the protection of sensitive data.

In summary, related work on CosmosDB spans various dimensions, including its multi-model capabilities, global distribution, consistency models, performance benchmarks, real-world applications, query processing, cost considerations, and security features. This collective body of knowledge contributes to a comprehensive understanding of CosmosDB's capabilities and considerations for effective use in diverse and globalized scenarios.

III. Experimental Setup

There are four stages carried out in this study, namely: system analysis, identification of hardware & software requirements, design, implementation, and measurement.

A. System Analysis

The system analysis for the performance evaluation of document databases involves a detailed examination of prominent NoSQL systems such as MongoDB. The primary focus is on assessing their capabilities in managing millions of queries, emphasizing key metrics like throughput, latency, and scalability.

B. Identification of Hardware and Software Needs

Hardware and software identification is needed to process the response time of the query. The hardware used for the demonstration is mentioned in the below table.

TABLE I
HARDWARE SPECIFICATION

No	Item	Description
1	Server	- AMD Processor: AMD R5-5600H Hexa Core - 8GB RAM - 4 GB NVIDIA GeForce GTX 1650 Graphics

As far as hardware is required some software is also required to perform the test. The software specification required for the experiment is mentioned in Table 2.

TABLE II
SOFTWARE SPECIFICATION

No	Software	Version
1	Java	1.8.0_382
2	Maven	3.6.3
3	Docker	24.0.7
4	Prometheus	2.48.0
5	Grafana	8.5.3
6	Ubuntu	22.04
7	MongoDB	7.0.3

C. Designing

At this stage, technical planning is completed, and preparations are in place before commencing the technical measurement process.

1) Data Prepration: The data utilized for analysis was generated through a Python script, comprising three fields: id, name, and value.

```
import json
import random
import string

def generate_random_data(record_count):
    data = []
    used_ids = set()

    for _ in range(record_count):
        # Generate a unique ID
        unique_id = generate_unique_id(used_ids)

        record = {
            'id': unique_id,
            'name': ''.join(random.choices(string.
            'value': random.uniform(0, 100)
        }

        data.append(record)
        used_ids.add(unique_id)

    return data

def generate_unique_id(used_ids):
    # Generate a random ID until a unique one is fo
    while True:
        unique_id = random.randint(1, 1000000)
        if unique_id not in used_ids:
            return unique_id

def save_to_json_file(data, file_path):
    with open(file_path, 'w') as json_file:
        json.dump(data, json_file, indent=2)

# Generate 1 million records
million_records = generate_random_data(1000000)

# Save data to a JSON file
save_to_json_file(million_records, 'generated_data
```

2) Experimental Design: The insertion operation was executed twice, employing distinct code approaches: direct insertion and parallel processing with batch processing. The dataset generated in the data preparation stage, comprising 1 million records, served as the basis for this comparative evaluation.

3) Mesurement: Measuring the response time for each query provides a direct and effective means to assess the comparative efficiency of different methods in our Spring Boot application interacting with MongoDB.

D. Implementation and Mesurement

The current setup involves the installation of MongoDB on prepared hardware. To monitor and gather metrics from MongoDB, Prometheus is installed via Docker, providing a centralized collection point for metrics. Additionally, Grafana, also installed via Docker, is configured to query the metrics collected by Prometheus, facilitating comprehensive visualization.

For seamless integration with this monitoring setup, the Actuator dependency is injected into the Spring Boot application. This allows the application to expose MongoDB-related metrics, which are subsequently ingested by Prometheus and made available for visualization in Grafana. By utilizing this end-to-end monitoring stack, the system gains the capability to capture, store, and visualize MongoDB performance metrics,

providing valuable insights for ongoing performance analysis and optimization efforts. This integrated approach ensures a streamlined and efficient monitoring environment for the MongoDB-backed Spring Boot application.

IV. Experimental Result

A. Code Preparation

The initial stage encompasses crafting the code logic to execute the insertion operation. As illustrated in Figure 1 below, the provided code block generally performs individual insert operations sequentially using a single connection pool.

```
public void insertDataFromJsonFiles(String json) {
    try {
        ObjectMapper objectMapper = new ObjectMapper();
        List<Demo> demos = objectMapper.readValue(json, new TypeReference<List<Demo>>() {});

        for (Demo demo : demos) {
            //repository.save(demo);
            dataInsertionTimer.record(() -> {repository.save(demo);});
            // Additional processing or metrics recording can be added here
        }
    } catch (IOException e) {
        logger.error("Error during data insertion: {}", e.getMessage());
        failedInsertionCounter.increment();
        // Handle the exception as needed
    }
}
```

Fig. 1. Normal Insertion

```
public void insertDataFromJsonFilesP(String json) {
    try {
        ObjectMapper objectMapper = new ObjectMapper();
        List<Demo> demos = objectMapper.readValue(json, new TypeReference<List<Demo>>() {});

        int batchSize = 1000; // Adjust the batch size as needed

        demos.parallelStream()
             .collect(Collectors.groupingByConcurrent(demo -> demos.indexOf(demo) / batchSize))
             .values()
             .parallelStream()
             .forEach(batch -> {
                 dataInsertionTimer.record(() -> {
                     repository.saveAll(batch);
                 });
                 // Additional processing or metrics recording can be added here
             });
    } catch (IOException e) {
        logger.error("Error during data insertion: {}", e.getMessage());
        failedInsertionCounter.increment();
        // Handle the exception as needed
    }
}
```

Fig. 2. Parallel Insertion

In Figure 2, optimization of the insertion process is achieved through parallel processing and batch processing. Here, batches of 1000 rows are created to be inserted simultaneously, leveraging the effectiveness of hardware by employing 6 parallel connection pools for concurrent operations.

B. Code Execution

The monitoring process is initiated by deploying Prometheus and Grafana images to capture and visualize metrics. Following this, the Spring application is launched, triggering API requests to the database for metric collection and analysis. The normal insertion process is activated by making a POST request to the standard API call, as depicted in Figure 3. This call involves sending a file containing 1 million records, which is then processed by the code block illustrated in Figure 4.

Similarly, following the completion of the first request, a second request is made. This time, a POST call is directed to

```
vinay@vinay:~$ curl -X POST -F "file=@/media/vinay/Study/IIT/BigData/Project/Prom
etheus/generated_data.json" http://localhost:8080/insertData
```

Fig. 3. Normal API Call Command

```
@PostMapping("/insertData")
public ResponseEntity<String> insertData(@RequestParam("file") MultipartFile jsonFile) throws IOException {
    if (jsonFile.isEmpty()) {
        return new ResponseEntity<>("File is empty", HttpStatus.BAD_REQUEST);
    }

    try {
        byte[] bytes = jsonFile.getBytes();
        String jsonString = new String(bytes);
        service.insertDataFromJsonFiles(jsonString);
        return new ResponseEntity<>("Data inserted successfully", HttpStatus.OK);
    } catch (IOException e) {
        e.printStackTrace();
        return new ResponseEntity<>("Error processing the file", HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Fig. 4. Normal API Call

the parallel processing API, as illustrated in Figure 5. The file is subsequently processed by the parallel processing API to execute further operations.

```
vinay@vinay:~$ curl -X POST -F "file=@/media/vinay/Study/IIT/BigData/Project/Prom
etheus/generated_data.json" http://localhost:8080/insertDataParallel
```

Fig. 5. Parallel API Call Command

```
@PostMapping("/insertDataParallel")
public ResponseEntity<String> insertDataP(@RequestParam("file") MultipartFile jsonFile) throws IOException {
    if (jsonFile.isEmpty()) {
        return new ResponseEntity<>("File is empty", HttpStatus.BAD_REQUEST);
    }

    try {
        byte[] bytes = jsonFile.getBytes();
        String jsonString = new String(bytes);
        service.insertDataFromJsonFilesP(jsonString);
        return new ResponseEntity<>("Data inserted successfully", HttpStatus.OK);
    } catch (IOException e) {
        e.printStackTrace();
        return new ResponseEntity<>("Error processing the file", HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Fig. 6. Parallel API Call

C. Result

Upon completion of the execution, an in-depth analysis of the results uncovered a discernible decline in insertion rates. This degradation was attributed to the sequential management of queries utilizing a solitary connection pool. The impact became more pronounced, especially under elevated query loads, thereby highlighting the limitations of the current configuration in handling increased demands on database operations. The insertion rate trends are visually represented in Figure 8 for reference.

Examining Figure 10, depicting the performance of batch and parallel processing, it becomes evident that the insertion rate follows a linear trend, contrasting with the incremental nature of normal insertion. Furthermore, the queries executed in less time compared to normal execution, underscoring the efficiency gains attributed to parallel processing. This observation implies that the implementation of batch processing and parallelization has contributed to a more streamlined and expedited execution of database queries.

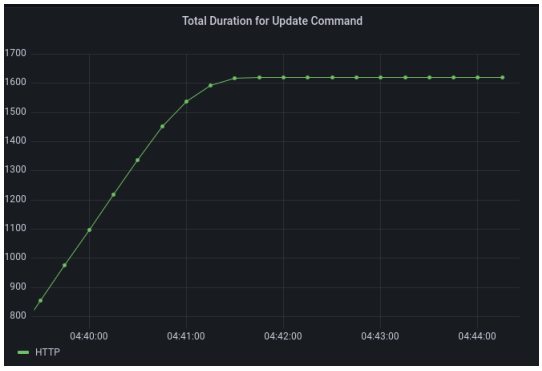

Fig. 7. Normal Insert Result

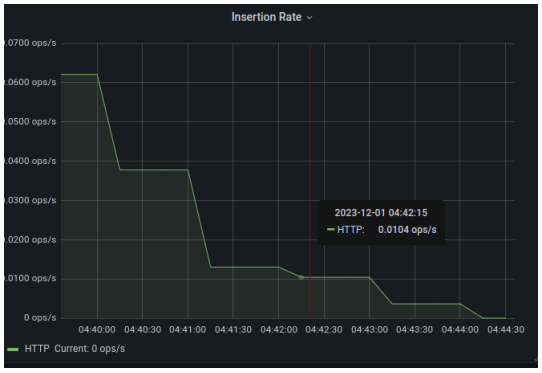

Fig. 8. Normal Insertion Rate

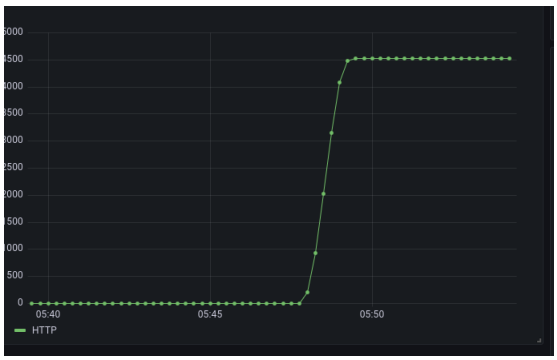

Fig. 9. Parallel Execution Result

V. Conclusion

In conclusion, our comprehensive performance evaluation of MongoDB, utilizing various insertion methods, has provided valuable insights into the efficiency of database operations. The analysis revealed that traditional sequential insertion exhibited limitations, especially under heightened query loads, leading to a noticeable decline in insertion rates. However, the adoption of parallel processing and batch insertion methods showcased a linear insertion rate and significantly reduced query execution times.

The results underscore the importance of optimizing database operations to align with the demands of contemporary applications. By leveraging parallel processing and batch techniques, our findings suggest a notable enhancement in both insertion efficiency and overall query performance. This implies that strategic implementation of these techniques can significantly contribute to the scalability and responsiveness of MongoDB-backed systems.

Fig. 10. Parallel Processing Rate

In future implementations, considering the effectiveness demonstrated by parallel processing and batch insertion, the adoption of these methodologies should be explored to harness their potential benefits in real-world applications. This study serves as a foundation for informed decision-making in optimizing MongoDB performance and contributes to the broader discourse on enhancing database operations in dynamic and data-intensive environments.

References

[1] Shin Morishima, Hiroki Matsutani: Performance Evaluations of Document-Oriented Databases using GPU and Cache Structure
[2] Ciprian-Octavian Truică , Florin Rădulescu , Alexandru Boicea, and Ion Bucur: Performance evaluation for CRUD operations in asynchronously replicated document oriented database
[3] Inês Carvalho, Filipe Sá and Jorge Bernardino: Performance Evaluation of NoSQL Document Databases: Couchbase, CouchDB, and MongoDB Transactions on Visualization and Computer Graphics, 2017.
[4] Rohmat Gunawan, Alam Rahmatulloh, Irfan Darmawan: Performance Evaluation of Query Response Time in The Document Stored NoSQL Database
[5] Yunhua Gu, Shu Shen, Jin Wang, Jeong-Uk Kim: Application of NoSQL Database MongoDB